

Linguagem C++ - Coleções: Containers Sequenciais - Fila Dupla, Lista e Lista Progressiva

Leonardo Medeiros

2025

Objetivos de Aprendizado

- Depois dessa unidade você deve ser capaz de:
 - Usar fila dupla, lista e lista progressiva em C++

Tópicos

- Containers Sequenciais
 - `std::deque`
 - `std::list`
 - `std::forward_list`

Container `std::deque`

- Deque é um acrônimo para Double Ended Queue (Fila Dupla)
- Sequencial e dinâmico (tamanho muda em tempo de execução)
- Permite acesso direto
- Dados são inseridos e removidos nas pontas
- `std::deque` é definido no cabeçalho `<deque>`

```
#include <deque>
```

Sintaxe do `std::deque`

```
std::deque<tipo> nome {};
```

Exemplo de Declaração do `std::deque`

```
std::deque<int> deq;
```

- Com inicialização:

```
deque<string> cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

Acesso ao `std::deque`

- Podemos usar o índice dentro do operador `[]`

```
cout << cars[0]; // Outputs Volvo  
cars[0] = "Opel";
```

- As funções `.front()` ou `.back()` para acessar o primeiro ou último elemento

```
cout << cars.front();  
cout << cars.back();
```

- Ou a função `.at()` para uma posição específica

```
cout << cars.at(2);  
cars.at(0) = "Opel";
```

Adicionar elementos aos `std::deque`

- Adicione elementos com `.push_front()` ou `.push_back()`

```
cars.push_front("BYD");  
cars.push_back("VW");
```

Remover elementos aos std::deque

- Remova elementos com `.pop_front()` ou `.pop_back()`

```
cars.pop_front();  
cars.pop_back();
```

Exemplo do `std::deque`

```
#include <iostream>
#include <deque>

int main()
{
    std::deque<int> deq;
    for (int count=0; count < 3; ++count)
    {
        deq.push_back(count); // insert at end of array
        deq.push_front(10 - count); // insert at front of array
    }

    for (int index=0; index < deq.size(); ++index)
        std::cout << deq[index] << ' ';

    std::cout << '\n';
}
```

- Este programa produz o resultado: 8 9 10 0 1 2

Container `std::list`

- `std::list` implementa uma Lista Duplamente Encadeada
- Sequencial e dinâmico (tamanho muda em tempo de execução)
- Não permite acesso direto
- Dados são inseridos e removidos nas pontas
- `std::list` é definido no cabeçalho `<list>`

```
#include <list>
```

Sintaxe do `std::list`

```
std::list<tipo> nome {};
```

Exemplo de Declaração do `std::list`

```
std::list<int> lista;
```

- Com inicialização:

```
list<string> cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

Acesso ao `std::list`

- Não podemos usar o índice dentro do operador `[]`, como no array, vector ou deque
- As funções `.front()` ou `.back()` acessam o primeiro ou último elemento

```
cout << cars.front();  
cout << cars.back();
```

Adicionar elementos aos `std::list`

- Adicione elementos com `.push_front()` ou `.push_back()`

```
cars.push_front("BYD");  
cars.push_back("VW");
```

Remover elementos aos `std::list`

- Remova elementos com `.pop_front()` ou `.pop_back()`

```
cars.pop_front();  
cars.pop_back();
```

Exemplo do `std::list`

```
#include <iostream>
#include <list>

int main()
{
    std::list<int> lista;
    for (int c=0; c < 3; ++c)
    {
        lista.push_back(c); // inserir no fim da lista
        lista.push_front(10 - c); // inserir no início da lista
    }
    for (int n : lista) {
        std::cout << n << " ";
    }
    std::cout << " | ";
    for (auto it = lista.rbegin(); it != lista.rend(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << '\n';
}
```

Exemplo do `std::list` (II)

- Este programa produz o resultado: 8 9 10 0 1 2 | 2 1 0 10 9 8

Container `std::forward_list`

- `std::forward_list` implementa uma Lista Simplesmente Encadeada
- Sequencial e dinâmico (tamanho muda em tempo de execução)
- Não permite acesso direto
- Dados são inseridos e removidos nas pontas
- `std::forward_list` é definido no cabeçalho `<forward_list>`

```
#include <forward_list>
```

Sintaxe do `std::forward_list`

```
std::forward_list<tipo> nome {};
```

Exemplo de Declaração do `std::forward_list`

```
std::forward_list<int> lista;
```

- Com inicialização:

```
forward_list<string> cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

Acesso ao `std::forward_list`

- Não podemos usar o índice dentro do operador `[]`, como no array, vector ou deque
- As funções `.front()` acessam o primeiro elemento

```
cout << cars.front();
```

Adicionar elementos aos `std::forward_list`

- Adicione elementos com `.push_front()`

```
cars.push_front("BYD");  
cars.push_back("VW");
```

Remover elementos aos `std::forward_list`

- Remova elementos com `.pop_front()`

```
cars.pop_front();
```

Exemplo do `std::forward_list`

```
#include <iostream>
#include <forward_list>

int main()
{
    std::forward_list<int> lista;
    for (int c=0; c < 3; ++c)
    {
        lista.push_front(10 - c); // inserir no início da lista
    }
    for (int n : lista) {
        std::cout << n << " ";
    }
    std::cout << '\n';
}
```

- Este programa produz o resultado: 8 9 10

Sumário, Atividades e Dúvidas

- `std::deque`, `std::list` e `std::forward_list`
- Atividade 1: Escreva os seguintes programas em C++.
 - Defina um `std::deque` para armazenar os 6 primeiros números pares e exibir os elementos.
 - Defina um `std::list` para armazenar os 6 primeiros números pares e exibir os elementos na ordem direta e reversa.
 - Defina um `std::forward_list` para armazenar os 6 primeiros números pares e exibir os elementos.
- Perguntas?
- Anotem e tragam as dúvidas na próxima aula

Referências

- Learn C++. <https://www.learncplusplus.com/>

Obrigado!